

Despliegue y Arquitectura de Infraestructura: FurMatch Frontend

Este documento detalla cómo está estructurada y automatizada la infraestructura de despliegue para la aplicación Frontend de FurMatch en Azure.

1. Arquitectura de Despliegue

El frontend utiliza una arquitectura moderna basada en **contenedores** y una canalización de **Integración y Despliegue Continuo (CI/CD)**. La aplicación está desarrollada en **Next.js** y se despliega como un servidor de Node.js optimizado (Standalone) dentro de un contenedor Docker.

Componentes Principales:

1. **Código Fuente:** Alojado en GitHub.
2. **CI/CD:** Automatizado a través de GitHub Actions ([.github/workflows/frontend-auth.yml](#)).
3. **Registro de Contenedores:** GitHub Container Registry (GHCR) actúa como repositorio central donde se almacenan y versionan las imágenes Docker compiladas.
4. **Hospedaje (Hosting): Azure App Service for Containers (Linux)** descarga la imagen Docker compilada de GHCR y la ejecuta en un entorno escalable y administrado por Microsoft Azure.

2. El Contenedor (Docker)

La aplicación está contenerizada utilizando un **Dockerfile** con una estrategia de **Construcción Multi-Etapa (Multi-Stage Build)**. Esto permite separar las dependencias de desarrollo y el proceso de construcción de lo que realmente se ejecuta en producción, resultando en una imagen final extremadamente ligera.

- **Etapa 1 (deps):** Se encarga únicamente de descargar dependencias instalando lo que detalla el [package-lock.json](#).
- **Etapa 2 (builder):** Toma los archivos fuente, inyecta variables de entorno requeridas para el *build* ([NEXT_PUBLIC_API_URL](#), etc.) y corre el comando `next build`, generando una versión optimizada en `.next/standalone`.
- **Etapa 3 (runner):** Construye la imagen de producción. Copia *exclusivamente* los estáticos y los archivos generados bajo el formato *standalone*. Introduce un usuario de seguridad (`nextjs` con UID 1001) para que el servidor no corra como `root`, una práctica fundamental de ciberseguridad.

Puerto Interno: El contenedor expone y escucha peticiones internamente a través del puerto **8080**.

3. Flujo Automático de Integración y Entrega (CI/CD)

El archivo [.github/workflows/frontend-auth.yml](#) define dos *Jobs* principales que se disparan de forma automática **cada vez que se hace push a la rama `deploy`**:

Job 1: **build-and-push**

1. Autentica GitHub Actions en el GitHub Container Registry (**ghcr.io**).
2. Transforma el nombre del propietario del repositorio a minúsculas, requerido por las convenciones estrictas de Docker para las etiquetas de las imágenes.
3. Construye un nuevo contenedor de Docker basándose en el **.** (raíz).
4. Sube (Push) la imagen recién compilada etiquetada bajo **ghcr.io/<owner>/furmatch-frontend:latest** reemplazando la versión anterior.

Job 2: **deploy**

**(Este Job requiere que el Job 1 se ejecute con éxito primero)*

1. Autentica contra Azure usando un Service Principal guardado de manera segura en el secreto **AZURE_CREDENTIALS** del repositorio.
2. Utiliza la acción oficial de Azure **azure/webapps-deploy** para instruir al portal web (**app-adopcion-frontend-dev**) que reinicie y descargue la imagen más reciente (**latest**) directamente desde el GitHub Container Registry.

4. Configuración en Azure (App Service)

Para que el ecosistema funcione de manera fluida y Azure sepa cómo interpretar la imagen obtenida de GitHub, la Web App en Azure (**app-adopcion-frontend-dev**) requiere la siguiente configuración en la nube:

Configuraciones de Entorno (App Settings)

Variable	Propósito
WEBSITES_PORT	8080 (<i>Obligatorio</i>). Le indica al balanceador de carga de Azure que enrute el tráfico HTTP/HTTPS externo hacia el puerto 8080 dentro del contenedor de Docker.
NEXT_PUBLIC_API_URL	Define la URL hacia el servidor backend con el cual se comunica y hace consumo de los endpoints la aplicación Frontend.

Redes de Confianza y Extracción

Debido a que el registro (GHCR) puede ser privado o público temporalmente, Azure App Service en su panel de "Deployment Center" (Centro de Implementación) también debe estar configurado para autenticarse si el contenedor es privado (usando un Token de acceso personal generado en GitHub con permisos **read:packages**).

5. Resumen del Flujo Completo

1. **Desarrollador** escribe código y hace **git push origin deploy**.
2. **GitHub Actions** detecta el cambio e inicia el entorno efímero.

3. **Docker** lee el **Dockerfile**, compila el App en Next.js, genera una mini-máquina virtual con Node 20 y almacena la nueva imagen.
4. Esa imagen viaja a **GHCR**, obteniendo la etiqueta **:latest**.
5. **Azure** recibe la señal, detiene el contenedor anterior y descarga/levanta el nuevo desde GHCR respondiendo en el puerto 8080.
6. La aplicación está online